CONTROL STRUCTURES: SWITCH CASES



LECTURE OUTLINE

Ternary Operators

Switch in C++

CONDITIONAL EXPRESSION

- Ternary operators
  - if (x>0)
  - {
  - Y=1;
  - }
  - else
  - {
  - Y=-1;
  - }
- The above code can be written as follows using ternary operators
- Y = x > 0 ? 1 : -1;
- GENERIC FORM : condition ? Value_if_true: value_if_false

CONDITIONAL EXPRESSION

- Task: Ask user to enter two numbers assign greater number to max

  max = num1 > num2 ? num1 : num2;

- Task: Ask user to enter a number find if it is even or odd using ternary operators

```
cout<<(num % 2==0 ? “Even” : “ Odd”)<<endl;
```

SWITCH STATEMENTS

- *A* switch *statement executes statements based on the value of a variable or an expression.*

  Here is the full syntax for the switch statement

```
switch (switch-expression)
{
  case value1: statement(s)1;
               break;
  case value2: statement(s)2;
               break;
  ...
  case valueN: statement(s)N;
               break;
  default:     statement(s)-for-default;
}
```

SWITCH STATEMENTS

- The switch statement observes the following rules:
  - ✓The switch-expression must yield an integral value and always be enclosed in parentheses.
  - ✓The value1, . . . , and value N are integral constant expressions, meaning that they cannot contain variables, such as 1 + x. These values are integers and cannot be floating-point values.
  - ✓When the value in a case statement matches the value of the switch-expression, the statements *starting from this case* are executed until either a break statement or the end of the switch statement is reached.
  - ✓The default case, which is optional, can be used to perform actions when none of the specified cases matches the switch-expression.
  - ✓The keyword break is optional. The break statement immediately ends the switch statement.

SWITCH STRUCTURES

- switch structure: alternate to if-else
- switch (integral) expression is evaluated first
- Value of the expression determines which corresponding action is taken
- Expression is sometimes called the selector

```
switch (expression)
{
case value1:
    statements1
    break;
case value2:
    statements2
    break;
    .
    .
    .
case valuen:
    statementsn
    break;
default:
    statements
}
```

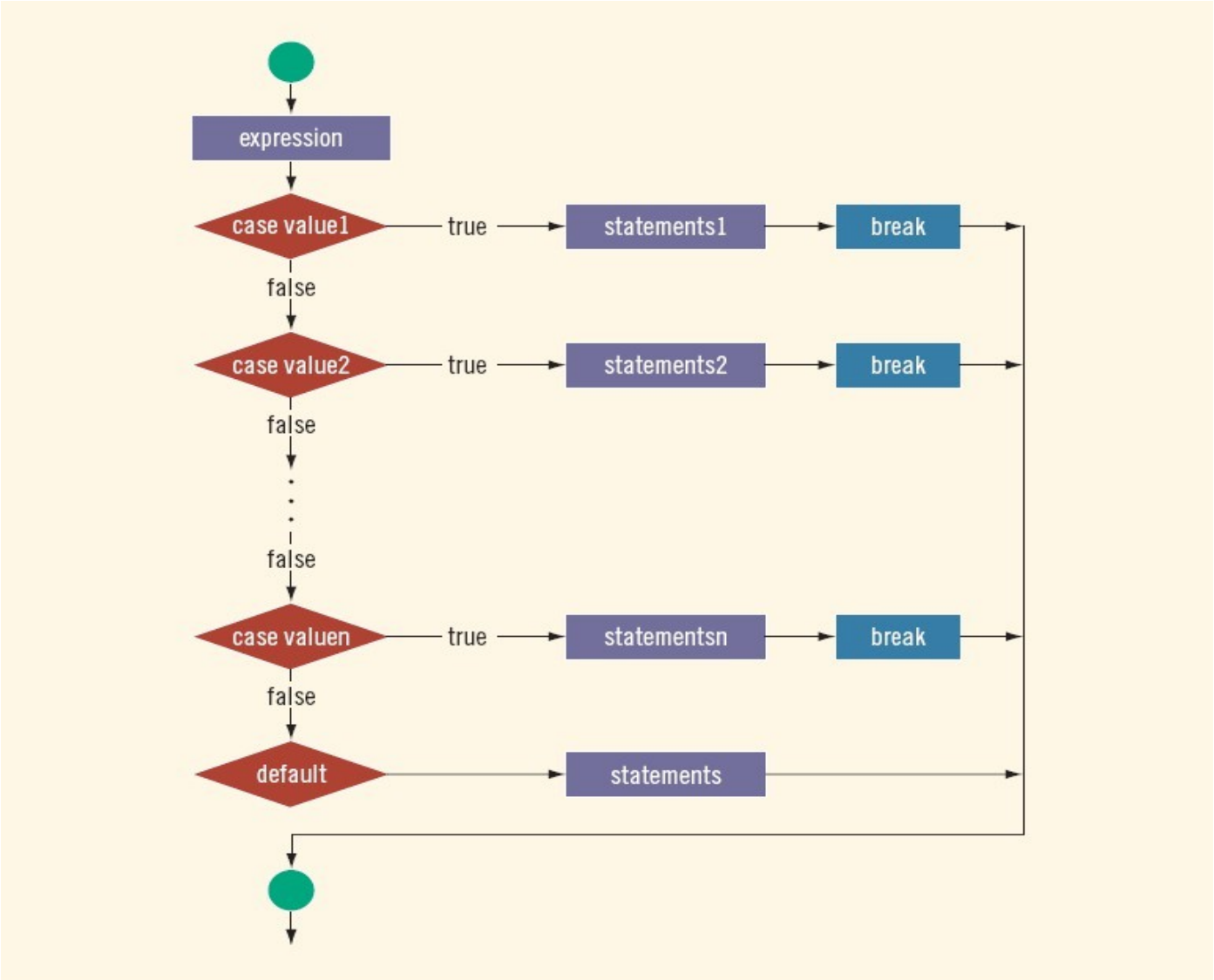


FIGURE 4-4 `switch` statement

`switch` STRUCTURES (CONTINUED)

- One or more statements may follow a case label
- Braces are not needed to turn multiple statements into a single compound statement
- The `break` statement may or may not appear after each statement
- `switch`, `case`, `break`, and `default` are reserved words

EXAMPLE 4-24

Consider the following statements, where grade is a variable of type `char`:

```
switch (grade)
{
case 'A':
    cout << "The grade is 4.0.";
    break;
case 'B':
    cout << "The grade is 3.0.";
    break;
case 'C':
    cout << "The grade is 2.0.";
    break;
case 'D':
    cout << "The grade is 1.0.";
    break;
case 'F':
    cout << "The grade is 0.0.";
    break;
default:
    cout << "The grade is invalid.";
}
```

In this example, the expression in the `switch` statement is a variable identifier. The variable `grade` is of type `char`, which is an integral type. The possible values of `grade` are `'A'`, `'B'`, `'C'`, `'D'`, and `'F'`. Each `case` label specifies a different action to take, depending on the value of `grade`. If the value of `grade` is `'A'`, the output is:

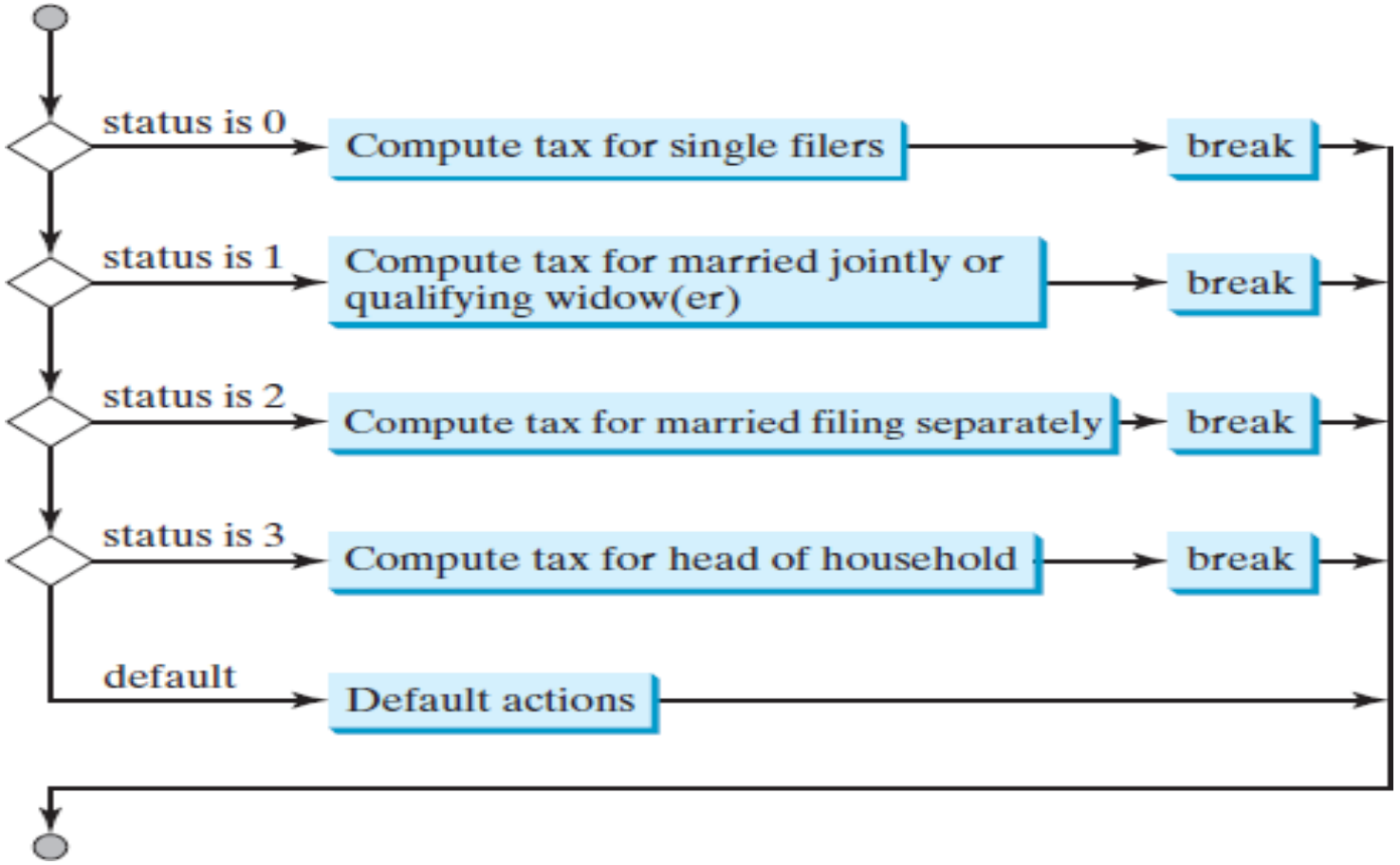
```
The grade is 4.0.
```

SWITCH STATEMENTS

- Lets see the flow chart for the following switch

```
switch (status)
{
  case 0:  compute tax for single filers;
           break;
  case 1:  compute tax for married jointly or qualifying widow(er);
           break;
  case 2:  compute tax for married filing separately;
           break;
  case 3:  compute tax for head of household;
           break;
  default: cout << "Error: invalid status" << endl;
}
```

FLOW CHART


status is 0
Compute tax for single filers
break
status is 1
Compute tax for married jointly or qualifying widow(er)
break
status is 2
Compute tax for married filing separately
break
status is 3
Compute tax for head of household
break
default
Default actions

WITHOUT BREAK WHAT WILL HAPPEN

```
#include<iostream>
#include<cstdlib>
using namespace std;
main()
{
      int days;
      cout<<" enter value 1 to 7 \n";
      cin>>days;
      if(days>=1&&days<=7)
      {
            switch (days)
            {
                  case 1:
                        cout<<" Monday "<<endl;
                  case 2:
                        cout<<" Tuesday "<<endl;
                  case 3:
                        cout<<" Wednesday "<<endl;
                  case 4:
                        cout<<" Thursday "<<endl;
                  case 5:
                        cout<<" Friday"<<endl;
                  case 6:
                        cout<<" Saturday "<<endl;
                  case 7:
                        cout<<" Sunday "<<endl;
                  default:
                        cout<<" Invalid Input"<<endl;

            }

      }
}
```

```
 enter value 1 to 7
2
 Tuesday
 Wednesday
 Thursday
 Friday
 Saturday
 Sunday
 Invalid Input
```

WITHOUT BREAK WHAT WILL HAPPEN

- Do not forget to use a break statement when necessary. Once a case is matched, the statements starting from the matched case are executed until a break statement or the end of the switch statement is reached. This is called *fall-through behavior*

NOW LETS SEE WHAT HAPPENS AFTER INSERTING BREAK IN EACH CASE

```
#include<iostream>
#include<cstdlib>
using namespace std;
main()
{
     int days;
     cout<<" enter value 1 to 7 \n";
     cin>>days;
     if(days>=1&&days<=7)
     {
          switch (days)
          {
               case 1:
                    cout<<" Monday "<<endl;
                    break;
               case 2:
                    cout<<" Tuesday "<<endl;
                    break;
               case 3:
                    cout<<" Wednesday "<<endl;
                    break;
               case 4:
                    cout<<" Thursday "<<endl;
                    break;
               case 5:
                    cout<<" Friday"<<endl;
                    break;
               case 6:
                    cout<<" Saturday "<<endl;
                    break;
               case 7:
                    cout<<" Sunday "<<endl;
                    break;
               default:
                    cout<<" Invalid Input"<<endl;

          }

     }
}
```

```
 enter value 1 to 7
2
 Tuesday
```

TERMINATING A PROGRAM WITH THE `ASSERT` FUNCTION

- Certain types of errors that are very difficult to catch can occur in a program
  - – Example: division by zero can be difficult to catch using any of the programming techniques examined so far
- The predefined function, `assert`, is useful in stopping program execution when certain elusive errors occur

THE ASSERT FUNCTION (CONTINUED)

- Syntax:

```
assert(expression);
```

  `expression` is any logical expression
- If `expression` evaluates to `true`, the next statement executes
- If `expression` evaluates to `false`, the program terminates and indicates where in the program the error occurred
- To use `assert`, include `cassert` header file

THE `ASSERT` FUNCTION (CONTINUED)

- `assert` is useful for enforcing programming constraints during program development
- After developing and testing a program, remove or disable assert statements
- The preprocessor directive `#define NDEBUG` must be placed before the directive `#include <cassert>` to disable the assert statement

PROGRAMMING EXAMPLE: CABLE COMPANY BILLING

- This programming example calculates a customer's bill for a local cable company
- There are two types of customers:
  - Residential
  - Business
- Two rates for calculating a cable bill:
  - One for residential customers
  - One for business customers

PROGRAMMING EXAMPLE: RATES

- For residential customer:
  - Bill processing fee: $4.50
  - Basic service fee: $20.50
  - Premium channel: $7.50 per channel
- For business customer:
  - Bill processing fee: $15.00
  - Basic service fee: $75.00 for first 10 connections and $5.00 for each additional connection
  - Premium channel cost: $50.00 per channel for any number of connections

PROGRAMMING EXAMPLE: REQUIREMENTS

- Ask user for account number and customer code
- Assume `R` or `r` stands for residential customer and `B` or `b` stands for business customer

PROGRAMMING EXAMPLE: INPUT AND OUTPUT

- Input:
  - Customer account number
  - Customer code
  - Number of premium channels
  - For business customers, number of basic service connections
- Output:
  - Customer's account number
  - Billing amount

PROGRAMMING EXAMPLE: PROGRAM ANALYSIS

- Purpose: calculate and print billing amount
- Calculating billing amount requires:
  - Customer for whom the billing amount is calculated (residential or business)
  - Number of premium channels to which the customer subscribes
- For a business customer, you need:
  - Number of basic service connections
  - Number of premium channels

PROGRAMMING EXAMPLE: PROGRAM ANALYSIS (CONTINUED)

- Data needed to calculate the bill, such as bill processing fees and the cost of a premium channel, are known quantities
- The program should print the billing amount to two decimal places

PROGRAMMING EXAMPLE: ALGORITHM DESIGN

- Set precision to two decimal places
- Prompt user for account number and customer type
- If customer type is `R` or `r`
  - Prompt user for number of premium channels
  - Compute and print the bill
- If customer type is `B` or `b`
  - Prompt user for number of basic service connections and number of premium channels
  - Compute and print the bill

PROGRAMMING EXAMPLE: VARIABLES AND NAMED CONSTANTS

```
int accountNumber;  //variable to store the customer's
                    //account number
char customerType;     //variable to store the customer code
int numOfPremChannels; //variable to store the number
                       //of premium channels to which the
                       //customer subscribes
int numOfBasicServConn; //variable to store the
                     //number of basic service connections
                     //to which the customer subscribes
double amountDue;  //variable to store the billing amount
```

```
      //Named constants - residential customers
const double RES_BILL_PROC_FEES = 4.50;
const double RES_BASIC_SERV_COST = 20.50;
const double RES_COST_PREM_CHANNEL = 7.50;

      //Named constants - business customers
const double BUS_BILL_PROC_FEES = 15.00;
const double BUS_BASIC_SERV_COST = 75.00;
const double BUS_BASIC_CONN_COST = 5.00;
const double BUS_COST_PREM_CHANNEL = 50.00;
```

PROGRAMMING EXAMPLE: FORMULAS

Billing for residential customers:

```
amountDue = RES_BILL_PROC_FEES +
            RES_BASIC_SERV_COST
            + numOfPremChannels *
            RES_COST_PREM_CHANNEL;
```

PROGRAMMING EXAMPLE: FORMULAS (CONTINUED)

Billing for business customers:

```
if (numOfBasicServConn <= 10)
    amountDue = BUS_BILL_PROC_FEES +
                BUS_BASIC_SERV_COST
                + numOfPremChannels *
                  BUS_COST_PREM_CHANNEL;
else
    amountDue = BUS_BILL_PROC_FEES +
                BUS_BASIC_SERV_COST
                + (numOfBasicServConn - 10)
                 * BUS_BASIC_CONN_COST
               + numOfPremChannels *
                 BUS_COST_PREM_CHANNEL;
```

PROGRAMMING EXAMPLE: MAIN ALGORITHM

1. Output floating-point numbers in fixed decimal with decimal point and trailing zeros
   - Output floating-point numbers with two decimal places and set the precision to two decimal places
2. Prompt user to enter account number
3. Get customer account number
4. Prompt user to enter customer code
5. Get customer code

PROGRAMMING EXAMPLE: MAIN ALGORITHM (CONTINUE

6. If the customer code is `r` or `R`,
   - Prompt user to enter number of premium channels
   - Get the number of premium channels
   - Calculate the billing amount
   - Print account number and billing amount

PROGRAMMING EXAMPLE: MAIN ALGORITHM (CONTINUE

7. If customer code is `b` or `B`,
   - Prompt user to enter number of basic service connections
   - Get number of basic service connections
   - Prompt user to enter number of premium channels
   - Get number of premium channels
   - Calculate billing amount
   - Print account number and billing amount

PROGRAMMING EXAMPLE: MAIN ALGORITHM (CONTINUED)

- 8. If customer code is other than `r`, `R`, `b`, or `B`, output an error message

SUMMARY

- Control structures alter normal control flow
- Most common control structures are selection and repetition
- Relational operators: ==, <, <=, >, >=, !=
- Logical expressions evaluate to `1` (`true`) or `0`
  - (`false`)
- Logical operators: `!` (not), `&&` (and), `||` (or)

SUMMARY (CONTINUED)

- Two selection structures: one-way selection and two-way selection
- The expression in an `if` or `if...else` structure is usually a logical expression
- No stand-alone `else` statement in C++
  - Every `else` has a related `if`
- A sequence of statements enclosed between braces, `{` and `}`, is called a compound statement or block of statements

SUMMARY (CONTINUED)

- Using assignment in place of the equality operator creates a semantic error
- `switch` structure handles multiway selection
- `break` statement ends `switch` statement
- Use `assert` to terminate a program if certain conditions are not met